

Cambio a bases de datos del portal biblioteca en PHP

**de Joel Narvaez, Alicia Valera y Allison
Velasquez de 2º DAW**

Índice

MYSQL.....	3
Creación base de datos:.....	3
Tablas:.....	3
Entidad Relación:.....	4
CRUD:.....	4
SQLITE.....	8
Instalación.....	8
Creación de la base de datos:.....	9
Uso de la base de datos:.....	9
Consultar en sqlite:.....	10

MYSQL

Creación base de datos:

1. **La creación de tablas:** La creación de los diferentes JSON ha sido creada a mano a través de phpmyadmin ingresando el tipo de datos y longitud de cada uno de los campos y sus diferentes restricciones.
2. **Creación de datos:** Los datos de la base de datos se han insertado a través de la propia app con el CRUD haciendo uso del método insert pasándole como parámetros la tabla a la cual se realizará el insert y un array asociativo con los datos recogidos en el respectivo formulario(users,checkouts,books).

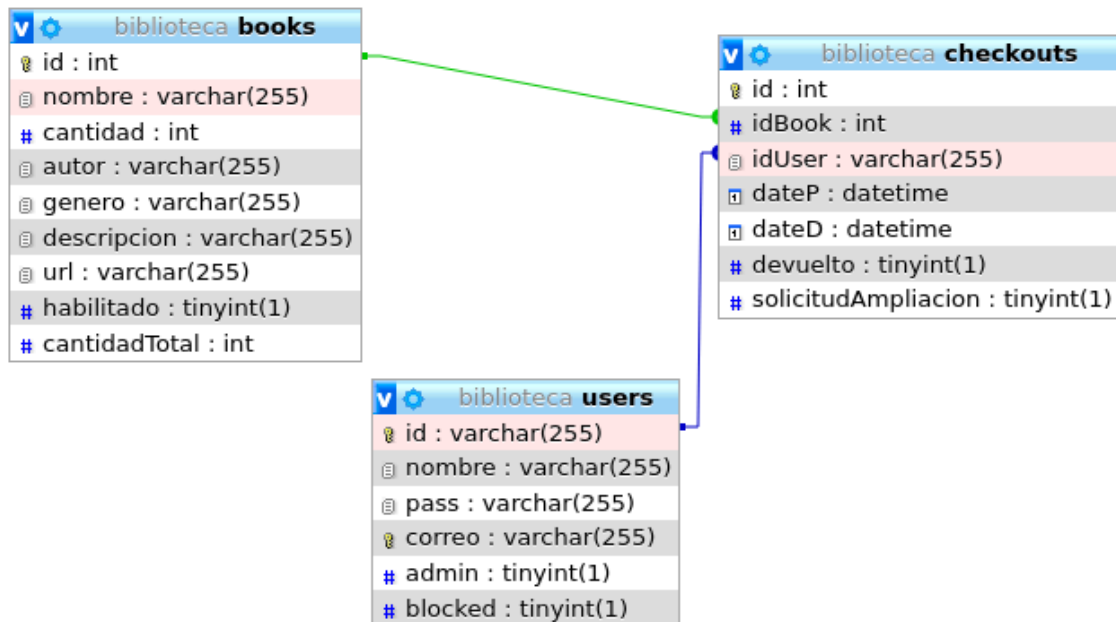
Tablas:

La creación de las tablas se realizó a través de la importación de la base de datos creada en phpmyadmin , especificando que se creen si no existen y en caso de existir no se ejecuta el create table .

```
CREATE TABLE IF NOT EXISTS `books` (  
  `id` int NOT NULL,  
  `nombre` varchar(255) NOT NULL,  
  `cantidad` int NOT NULL DEFAULT '1',  
  `autor` varchar(255) NOT NULL,  
  `genero` varchar(255) NOT NULL,  
  `descripcion` varchar(255) NOT NULL,  
  `url` varchar(255) NOT NULL,  
  `habilitado` tinyint(1) DEFAULT '1',  
  `cantidadTotal` int NOT NULL  
  
CREATE TABLE IF NOT EXISTS `checkouts` (  
  `id` int NOT NULL,  
  `idBook` int NOT NULL,  
  `idUser` varchar(255) NOT NULL,  
  `dateP` datetime NOT NULL,  
  `dateD` datetime NOT NULL,  
  `devuelto` tinyint(1) DEFAULT '0',  
  `solicitudAmpliacion` tinyint(1) DEFAULT '0'  
  
CREATE TABLE IF NOT EXISTS `users` (  
  `id` varchar(255) NOT NULL,  
  `nombre` varchar(255) NOT NULL,  
  `pass` varchar(255) NOT NULL,  
  `correo` varchar(255) NOT NULL,  
  `admin` tinyint(1) DEFAULT '0',  
  `blocked` tinyint(1) DEFAULT '0'  
);
```

Entidad Relación:

En el entidad relación podemos observar una relación entre las tres tablas a través del id de las tres tablas con una cardinalidad de (1:1:N) ya que los préstamos (checkouts) se asocian a un único id de usuario y a un único id de libro pero se pueden realizar múltiples préstamos del mismo libro pero se asocia a un diferente id en la tabla checkouts.



CRUD:

En el CRUD se han realizado las funciones básicas para manejar la transferencia de datos entre la aplicación y la base de datos, usando PDO

- **Config.php**

```
//Constantes de usuario para acceder a la Base de Datos
define("dsn", 'mysql:host=localhost;dbname=biblioteca');
define("dbuser", 'admin');
define("dbpass", 'admin');
```

- **Conectar**

El metodo conectar accede a los parámetros definidos en config.php que tiene los parámetros necesarios para crear la coneccion a la base de datos (host,user,password, nombre de la base de datos)

Devuelve la conexión o un mensaje de la excepción PDOException.

```
function conectar(): PDO
{
    try {
        $con = new PDO(dsn, dbuser, dbpass, [
            PDO::ATTR_ERRMODE,
            PDO::ERRMODE_EXCEPTION
        ]);

        return $con;
    } catch (PDOException $e) {
        echo 'error ' . $e->getMessage() . '<br>';
    }
}
```

- **GetAllByTable**

Este método recibe como parámetro el nombre de una tabla, realiza una llamada a conectar y realiza la consulta select a la tabla especificada y devuelve un array asociativo con los datos encontrados en la base de datos o un mensaje PDOException.

```
function getAllByTable($tabla): array
{
    try {
        $con = conectar();
        $sql = "SELECT * FROM $tabla";
        $data = $con->query($sql);

        return $data->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        echo 'error ' . $e->getMessage() . '<br>';
    }
}
```

- **GetData ByParam**

Método que recibe como parámetros: nombre de tabla, parámetro por el cual se realizará la búsqueda y el valor del parámetro.

Retorna un array asociativo con los datos que coinciden con la sentencia select en la base de datos o un mensaje de PDOException:

```
function getAllByParam($tabla, $param, $value): array
{
    try {
        $con = conectar();
        $sql = "SELECT * FROM $tabla WHERE $param='$value'";
        $data = $con->query($sql);
        return $data->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        echo 'error ' . $e->getMessage() . '<br>';
    }
}
```

- **GetDataById**

Método que recibe como parámetros: nombre de la tabla, parámetro a recuperar, id.

Realiza una sentencia select pidiendo el parámetro pasado y filtrando por id .

Retorna un array asociativo con los datos que coincidan con el id pasado por parámetro y en caso contrario retorna un mensaje de PDOException.

```
function getDataById($tabla, $param, $id): array
{
    try {
        $con = conectar();

        $sql = "SELECT $param FROM $tabla WHERE id = :id";
        $stmt = $con->prepare($sql);

        if (is_int($id)) {
            $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        } else {
            $stmt->bindParam(':id', $id, PDO::PARAM_STR);
        }

        $stmt->execute();

        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    } catch (PDOException $e) {
        echo 'Error: ' . $e->getMessage() . '<br>';
    }
}
```

- **DeleteById**

Método que recibe como parámetro : nombre de la tabla , id.

Ejecuta la sentencia delete filtrando por id, en caso de borrar algún dato retorna un true , en caso de no haber borrado ningún dato retorna un false y realiza un rollback y en caso de haber un error muestra el error a través del PDOException a través del metodo get->Message().

```
function deleteById($tabla, $id): bool
{
    try {
        $con = conectar();
        $con->beginTransaction();
        $consulta = "DELETE FROM $tabla WHERE id='$id'";
        $resultado = $con->exec($consulta);
        if ($resultado == 0) $con->rollback();
        if ($resultado != 0) {
            $con->commit();
            return true;
        }
        return false;
    } catch (PDOException $e) {
        echo 'Error ' . $e->getMessage() . '<br>';
    }
}
```

- **UpdateDataByParam**

Método que recibe como parámetros: nombre de la tabla, un array asociativo, condición y valor de la condición.

Recorre el array asociativo comprobando si el dato pasado es un string o un número y lo añade a otro arreglo con su respectiva estructura en caso de ser un string le añade las comillas al valor y en caso de no serlo lo añade directamente, realiza un implode para aplanar el arreglo y convertirlo en una cadena y se lo pasa a la sentencia update y añade el where con la condición pasada como parámetro con el condicional= valor condición.

Retorna un true en caso de afectar a alguna fila en la base de datos y realiza un commit para confirmar los cambios, un false en caso de no haber afectado a ninguna fila y un mensaje de PDOException en caso de que se produzca un error.

```
function updateDataByParam($tabla, $paramsYvalues, $condicion, $valueCondicion): bool
{
    try {
        $con = conectar();
        $con->beginTransaction();
        $newData = [];
        foreach ($paramsYvalues as $key => $value) {
            (is_string($value) ? $newData[] = "$key= '$value'" : $newData[] = "$key= $value");
        };
        $string = implode(',', $newData);
        $consulta = "UPDATE $tabla SET $string WHERE $condicion='$valueCondicion'";
        $resultado = $con->exec($consulta);
        if ($resultado == 0) $con->rollback();
        if ($resultado != 0) {
            $con->commit();
            return true;
        } else {
            return false;
        }
    } catch (PDOException $e) {
        echo 'Error ' . $e->getMessage() . '<br>';
    }
}
```

- **Insert**

Método que recibe como parámetro: nombre de la tabla y un array asociativo.

Prepara una transacción y recoge del array todos sus valores y lo almacena en otro arreglo, a partir del cual se cuenta cuántos valores posee y se crea una variable con tantos ? como valores posea el arreglo y con el implode se le añade una ',' y convierte el arreglo que devuelve en una cadena que se usa para realizar la sentencia preparada.

Recoge los parámetros con la función array_keys y los almacena en una variable para usar literalmente en la sentencia preparada insert.

Los valores se asignan con la función execute pasándole el array con los valores.

La función no retorna nada en caso de funcionar correctamente pero en caso de haber sucedido algún problema durante la ejecución retorna un mensaje PDOException.

```

function insert($tabla, $paramsYvalues): void
{
    try {
        $con = conectar();
        $con->beginTransaction();

        $valores = array_values($paramsYvalues);
        $column = implode(',', array_keys($paramsYvalues));

        $placeholders = implode(',', array_fill(0, count($valores), '?'));
        $senetencia = "INSERT INTO $tabla ($column) VALUES ($placeholders)";

        $stmt = $con->prepare($senetencia);
        $stmt->execute($valores);
        $con->commit();
    } catch (PDOException $e) {

        echo 'ERRRO' . $e->getMessage() . '<br>';
    }
}

```

SQLITE

Para realizar la migración de la base de datos a SQLITE es necesario tener instalado sqlite y tener en cuenta que tiene pequeñas variaciones:

Instalación

Se debe descargar desde <https://www.sqlite.org/download.html> el paquete tools para la versión que corresponda a nuestro sistema operativo y descomprimir la carpeta y añadir la ruta al path de las variables de entorno.

- **Create Database:** en sqlite no se maneja de la misma forma que en sql tanto las Bases de datos como los usuarios y en este caso los hemos suprimidos y creados directamente creando las tablas desde el documento .sql que se genera al exportar la base de datos desde phpMyAdmin modificando los tipos de datos y sustituyendo las partes que no son compatibles con y añadiendo la

```
PRAGMA foreign_keys = ON;
```

para poder hacer uso de las claves foráneas , También se eliminaron las siguientes líneas debido a que no son compatibles :

```

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
SET AUTOCOMMIT = 0;
START TRANSACTION;
SET time_zone = "+00:00";

```


- **Tipos de datos** :En sqlite son diferentes a sql

SQL	SQLITE
VARCHAR	TEXT
INT	INTEGER
TINYINT	INTEGER O BOOLEAN

Creación de la base de datos:

La base de datos se crea ejecutando en la terminal sqlite3 (nombre que se le quiere asignar a la base de datos).db < (ruta del archivo .sql) y se generara un archivo con los datos del archivo .sql.

Uso de la base de datos:

Para poder usarlo solo hemos tenido que modificar el metodo conectar cambiando los parámetros en el archivo config.php añadiendo una nueva constante con los datos necesarios para poder acceder a la base de datos de sqlite, añadimos la ruta de donde se crea la base de datos:

```
define("dsn", 'mysql:host=localhost;dbname=biblioteca');
define("dbuser", 'testuser');
define("dbpass", 'testpassword');

define("dsnSqlite", "sqlite:C:\\Users\\34672\\biblioteca.db");
```

Coneccion a la base de datos a través de Mysql

```
$con = new PDO(dsn, dbuser, dbpass, [
    PDO::ATTR_ERRMODE,
    PDO::ERRMODE_EXCEPTION
]);
```

Coneccion a la base de datos a través de sqlite.

```
$con = new PDO(dsnSqlite, '', '', [
    PDO::ATTR_ERRMODE,
    PDO::ERRMODE_EXCEPTION
]);
```

Consultar en sqlite:

1. Para poder ver las tablas se ejecuta sqlite3 con el nombre de la base de datos a la que queramos acceder y se nos abrirá un entorno interactivo en el cual podremos realizar la consulta de tablas ejecutando **.tables** que nos devolverá el nombre de las diversas tablas que tengamos en esa base de datos.
2. Para poder ver nuestras bases de datos se ejecuta dentro del entorno de sqlite .databases y nos mostrará el nombre y la ruta de todas las bases de datos activas en esa sesión , ya que en sqlite se puede tener mas de una base de datos activa con una session o también se puede ejecutar **database list**.
3. En sqlite las sentencias select, delete y update se realizan de la misma forma.

sql para crear tablas en sqlLite

```
CREATE TABLE IF NOT EXISTS `books` (  
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,  
  `nombre` TEXT NOT NULL,  
  `cantidad` INTEGER NOT NULL DEFAULT 1,  
  `autor` TEXT NOT NULL,  
  `genero` TEXT NOT NULL,  
  `descripcion` TEXT NOT NULL,  
  `url` TEXT NOT NULL,  
  `habilitado` INTEGER DEFAULT 1,  
  `cantidadTotal` INTEGER NOT NULL  
);  
  
INSERT INTO `books` (`id`, `nombre`, `cantidad`, `autor`, `genero`,  
`descripcion`, `url`, `habilitado`, `cantidadTotal`) VALUES  
(1, '1984', 1, 'George Orwell', 'Distopía', 'Una novela que describe un  
mundo totalitario donde el gobierno controla todos los aspectos de la  
vida humana.', '../img/1984.jpg', 1, 20),  
(2, 'El gran Gatsby', 6, 'F. Scott Fitzgerald', 'Ficción', 'La historia  
de Jay Gatsby y su obsesión con Daisy Buchanan en la era del jazz en  
los años 20.', '../img/el_gran_gatsby.jpeg', 1, 8),  
(3, 'Don Quijote de la Mancha', 8, 'Miguel de Cervantes', 'Clásicos',  
'Las aventuras de un caballero idealista y su fiel escudero, en una  
obra que explora la locura y la realidad.',  
 '../img/don_quijote_de_la_mancha.jpg', 1, 8),  
(4, 'Fahrenheit 451', 23, 'Ray Bradbury', 'Ciencia ficción', 'En un  
futuro distópico, los libros están prohibidos y los bomberos queman  
cualquier material literario encontrado.', '../img/fahrenheit_451.jpg',  
1, 25),  
(5, 'La sombra del viento', 10, 'Carlos Ruiz Zafón', 'Misterio', 'Un  
joven descubre un libro en un cementerio de libros olvidados, desatando
```

```

una serie de misteriosos eventos.', '../img/la_sombra_del_viento.jpg',
1, 9),
(6, 'Orgullo y prejuicio', 6, 'Jane Austen', 'Romance', 'La historia de
Elizabeth Bennet y su relación con el orgulloso señor Darcy en la
Inglaterra del siglo XIX.', '../img/orgullo_y_prejuicio.jpg', 1, 2),
(7, 'Matar a un ruiseñor', 5, 'Harper Lee', 'Ficción', 'Una historia
sobre la injusticia racial en el sur de Estados Unidos, vista a través
de los ojos de una niña.', '../img/matar_a_un_ruisenor.jpg', 1, 5),
(8, 'El alquimista', 5, 'Paulo Coelho', 'Ficción', 'La búsqueda de
Santiago, un joven pastor que viaja en busca de un tesoro personal, en
una alegoría sobre el destino.', '../img/el_alquimista.jpg', 1, 10);

```

```

CREATE TABLE IF NOT EXISTS `checkouts` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,
  `idBook` INTEGER NOT NULL,
  `idUser` TEXT NOT NULL,
  `dateP` DATETIME NOT NULL,
  `dateD` DATETIME NOT NULL,
  `devuelto` INTEGER DEFAULT 0,
  `solicitudAmpliacion` INTEGER DEFAULT 0,
  FOREIGN KEY (`idBook`) REFERENCES `books` (`id`) ON DELETE CASCADE,
  FOREIGN KEY (`idUser`) REFERENCES `users` (`id`) ON DELETE CASCADE
);

```

```

CREATE TABLE IF NOT EXISTS `users` (
  `id` TEXT PRIMARY KEY,
  `nombre` TEXT NOT NULL,
  `pass` TEXT NOT NULL,
  `correo` TEXT NOT NULL,
  `admin` INTEGER DEFAULT 0,
  `blocked` INTEGER DEFAULT 0
);

```

```

INSERT INTO `users` (`id`, `nombre`, `pass`, `correo`, `admin`,
`blocked`) VALUES
('admin', 'admin',
'$2y$10$yZmJlPRnOZPkzR9w6Bg8lujBnQ9s7cb1jdfF0qUv2NGjoXBRP8eFi',
'admin@admin.com', 1, 0);

```

```

PRAGMA foreign_keys = ON;
COMMIT;

```